

Mathematics of robot state and motion

M2 • 03 Nov 2026 - 29 Nov 2026 • 75 hours

What you should achieve by the end of M02

Learn to turn a simple leg or finger problem into quantities, equations and a verifiable calculation. For example: find a force's moment arm from a support point's position; recover an angle from encoder counts; obtain velocity from a trajectory; predict a compliant support's motion from mass, stiffness and damping. The main deliverable is **robot-math**, a collection of numerical and geometric checks reusable in later modules.

The scope is deliberately limited: vectors and matrices, linear systems and least squares, derivatives and integrals, and ordinary differential equations. Introduce the Jacobian as a table of sensitivities and eigenvalues as properties of a simple linear transformation and system. Do not yet implement full 3D robot kinematics, spatial velocities, controllers or a Kalman filter.

Starting model and preparation

Use Python, NumPy, Matplotlib and SciPy in the M01 environment. Create notebooks/01_vectors, 02_calibration, 03_motion and 04_odes; move repeated functions into robot_math/. Each notebook must contain a problem statement, axes diagram, parameters with units, calculation, error plot and conclusion. Keep raw and computed data separately; pin dependency versions and the random-number generator seed.

The shared teaching object is a link of length $L=0.12$ m with one revolute joint and a support point at its tip. Masses, forces and other numbers in this document are calculation-example parameters, not authorisation to test a future design. The second object is a compliant support with mass $m=0.5$ kg, stiffness $k=50$ N/m and viscous damping coefficient $c=1$ N·s/m. M02 investigates motion only through calculation.

Entry diagnostic: within the first two hours

Without references, convert 120 mm to metres, 90° to radians and 10 ms to seconds; find $\sin(\pi/2)$, a rectangle's area and the solution of two linear equations. In Python, create an array of shape (100,3), select one coordinate and calculate its mean. If unit arithmetic or trigonometry is difficult, spend up to 90 minutes of block A addressing that specific gap and reduce decorative plotting work.

Starting criterion: explain in your own words the difference between a number, a vector component and a physical quantity. Record one M01 error that mathematics could detect: a reversed axis sign, milliseconds used instead of seconds, or a false velocity jump from a missing packet. This becomes your first regression example.

Allocating 75 hours and recording calculations

Four study blocks

A - hours 1-20: 10 h of theory and hand calculations, 10 h of practice. Topics: units, vectors, bases, dot and cross products, matrices and rank. Deliverable: force diagrams and a library of verifiable point and vector transformations.

B - hours 21-40: 10 h of theory and 10 h of practice. Topics: Gaussian elimination, overdetermined systems, residuals, QR, conditioning and eigenvalues. Deliverable: encoder calibration with a separate validation set and an unsuccessful experiment example.

C - hours 41-60: 10 h of theory and 10 h of practice. Topics: derivatives, integrals, partial derivatives, gradients and an introductory Jacobian. Deliverable: link-trajectory analysis, work done by a force and sensitivity of the endpoint position to angle.

D - hours 61-75: 5 h of theory and 10 h of practice. Topics: initial-value problems, explicit Euler, semi-implicit Euler, RK4, stability and step-size error. Deliverable: mass-spring-damper, pendulum and convergence report. Total: exactly 35 h of analysis and 40 h of practice.

Session rhythm

Divide each 10 h of theory into five two-hour sessions: 15 min reviewing the preceding idea, 35 min reading, 40 min hand calculations, 20 min Python verification and 10 min recording conclusions. Theory in the final block takes 2+2+1 h. Divide the ten-hour practical into five 2 h sessions: formulation, implementation, cross-checking, error analysis and presentation. Breaks are outside study hours.

The calendar window on the first page comes from the roadmap. A boundary date may belong to two modules; rest days, holidays and integration stages remain as in the original plan. Numbered work steps fill available sessions sequentially without recounting hours. Do not add an extra hour to every module.

When to move from reading to practice

After each section, close the text and explain which quantity you are calculating, why the operation is valid and how you will verify the answer. One correct example and one deliberately faulty example are more useful than a second lecture on the same topic. Use these log fields: robot task; model; assumptions; prediction before running; result; error; next check.

If behind schedule, first remove animation styling and a second language implementation, then reduce parameter sweeps. Retain four notebooks, both dynamic objects, unit checks and method comparisons. Full QR proofs and derivation of RK4 from Taylor series can wait; explanations of residuals, time steps and causes of discrepancies are compulsory.

Vectors, bases and forces on a link

Topics to study

Study point and free-vector coordinates, length and normalisation, force addition, bases and projections. The dot product $\mathbf{a} \cdot \mathbf{b}$ connects components to angle; for unit direction $\mathbf{n}$, the force projection is $\mathbf{F} \cdot \mathbf{n}$. The cross product $\mathbf{r} \times \mathbf{F}$ gives torque about a chosen origin. Measure r in metres, F in newtons and torque in $\text{N} \cdot \text{m}$; its direction follows the right-hand rule.

Study matrix-vector multiplication as a coordinate transformation, matrix product order, transposition and the meaning of an inverse matrix. For planar rotation, use $R(\theta)$ with first row $[\cos\theta, -\sin\theta]$ and second row $[\sin\theta, \cos\theta]$. State the convention explicitly: an active anticlockwise vector rotation in a fixed frame. The reverse transformation uses $R(\theta)^T$; do not confuse the two operations.

Task 1. Loading a finger or shank

Input: $\mathbf{r} = (0.12; 0; 0)$ m, $\mathbf{F} = (0; 0; -5)$ N. First draw a side view and predict the torque direction. Then calculate $\mathbf{r} \times \mathbf{F}$ by hand and with NumPy, find its magnitude and project $\mathbf{F}$ onto the link axis. Rotate the force by 30° in the same plane and repeat. Deliverable: diagram and table of four positions, labelling axes, units and signs.

Check: in the first case, torque is $(0; 0.6; 0)$ $\text{N} \cdot \text{m}$; collinear $\mathbf{r}$ and $\mathbf{F}$ give zero. Doubling the moment-arm length must double torque magnitude. Faulty case: enter 120 instead of 0.12 and explain the exact thousandfold discrepancy. This checks the unit model; it is not a physical motor test.

Task 2. Sensor coordinates on a rotated body

Input: three unit vectors and ten arbitrary points in the plane. Implement R and rotate the set by 0° , 90° , 180° and back. For an offset-mounted sensor, distinguish $\mathbf{p}_{\text{body}} = R \cdot \mathbf{p}_{\text{sensor}} + \mathbf{t}$ from the direction transformation $\mathbf{v}_{\text{body}} = R \cdot \mathbf{v}_{\text{sensor}}$. Do not add translation $\mathbf{t}$ to force, acceleration or direction.

Save a plot of original and transformed axes, a norm test and a reverse-transformation test. For float64 numbers of order one, use a teaching absolute tolerance of 10^{-12} ; also explain the scale choice. Test the zero vector without normalising it and an angle in degrees mistakenly passed as radians. Be able to state by hand the result of rotating $(1, 0)$ by 90° .

Linear systems and encoder calibration

From equation to solution

Study $A \cdot x = b$: each row defines a constraint and each column an unknown's contribution. Work through Gaussian elimination on a 2×2 system, row swaps for a poor pivot, and nonexistent or nonunique solutions. Rank is the number of independent constraints. The inverse matrix is useful conceptually; solve numerically with `solve` or `lstsq` rather than `inverse(A) \cdot b`.

For calibration $q = a \cdot u + b$, u is the encoder count and q is in radians. Construct A from a column of u values and a column of ones. With more observations than parameters, minimise the sum of squared residuals $r = A \cdot x - q$. QR separates the problem into an orthogonal transformation and a triangular system; understand this role and one 3×2 example without writing a general QR implementation.

Task 3. A teaching body with two supports

A 2 kg body rests on two points at horizontal positions $x=0$ and $x=0.3$ m; its centre of mass is at $x=0.12$ m. Write vertical force and torque equilibrium equations; find support reactions by hand and in Python. With $g=9.81$ m/s², reactions must sum to 19.62 N, and opposing moments about the left support must balance.

Move the centre of mass from 0 to 0.3 m. A negative reaction outside the segment means the contact model, which can only push, is invalid. Do not present it as a feasible leg force. Save a reaction-force plot and mark the model's validity boundary; balance control comes later.

Task 4. Find encoder scale and zero

Create 21 u positions evenly spanning the teaching range $-1000 \dots 1000$. Generate q with known $a=0.001$ rad/count, $b=0.03$ rad and small noise with fixed settings; save the ground-truth parameters separately. Reserve some points for validation. Estimate the parameters, plot residual against u and calculate $RMSE = \sqrt{\text{mean}(r^2)}$ in radians. First check noise-free data: parameters must be recovered to machine precision.

Then restrict inputs to around $u=500$ and compare parameter sensitivity to a small change in one observation. A small error within a narrow region does not establish good calibration across the full range. Deliverables: `coefficients.json`, raw CSV, training and held-out error plots, rank of A and an explanation of units. Failure cases: identical u values, an empty set, NaN and a duplicate mistakenly counted as a new independent measurement.

Conditioning and eigenvector directions

Skills needed before M03 optimisation

The condition number describes how much input error can be amplified in the answer for a specified norm and scale. A large value warrants examining the formulation, units and data diversity. It is not a universal good/bad switch: parameters in millimetres and metres have different scales. Compare normalised formulations and always show the actual change in the answer.

Under a matrix transformation, an eigenvector remains on the same line; its eigenvalue is the multiplier. For a diagonal stiffness matrix $K = \text{diag}(40, 100)$ N/m, the directions are obvious. If off-diagonal terms couple the axes, compute the directions. For now, use a symmetric 2×2 matrix and check $K \cdot v = \lambda \cdot v$; leave complex modes, spectral control and general multilink mechanics for later months.

Task 5. Nearly identical support measurements

Use $A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + \epsilon \end{bmatrix}$ and b generated from known $x = (1, 2)$. Set $\epsilon = 10^{-1}, 10^{-4}, 10^{-8}$. For each case, save $\text{cond}(A)$, the residual and the error in recovered x . Add the same small relative perturbation to one b component and repeat. This illustrates two nearly duplicate constraints when determining a robot's state.

Check: a small residual can coexist with a large x error. Demonstrate this numerically, then replace the second equation with an independent one and repeat. Deliverable: six-run table and a conclusion identifying an additional measurement that resolves the problem. Do not insert a pseudoinverse without explanation: it belongs to M03 and cannot create missing information.

Task 6. Stiff and compliant support directions

For $K = \begin{bmatrix} 80 & 20 \\ 20 & 50 \end{bmatrix}$ N/m, use `eigh` to find eigenvalues and normalised eigenvectors. Draw original axes and eigenvector directions. Apply an equal teaching force along each direction and find displacement from $K \cdot x = F$. A greater stiffness λ produces less displacement for the same force magnitude.

Check orthogonality of the two directions, each eigen-equation's residual and reconstruction of the original matrix. Set stiffness along one axis to zero: this model no longer determines static displacement along the free direction. Save it as an expected solve failure caused by a singular system. Additional 20-minute experiment: rotate K using orthogonal R and verify unchanged eigenvalues.

Derivatives, integrals and motion sensitivity

Theory connected to motion

The time derivative of position is velocity; the derivative of velocity is acceleration. For angle q , units are rad, rad/s and rad/s². Study derivatives of sums, products and sin/cos, and the chain rule for $q(t)=A\cdot\sin(\omega t)$, with ω in rad/s. Integrating velocity gives coordinate change; integrating force along a path gives work in joules, and over time gives impulse in N·s. Label the integration variable for every integral.

A partial derivative describes how a function changes when one argument varies and the others remain fixed. The gradient collects these derivatives for a scalar function; a vector function's Jacobian arranges them with outputs in rows and inputs in columns. Do not call every multidimensional array a Jacobian: first specify the function, input/output dimensions and each element's units.

Task 7. Encoder velocity and noise

Generate 5 s of motion $q=0.4\cdot\sin(2\pi t)$ rad with a 0.01 s step. Derive exact velocity and acceleration. Implement forward and central differences; compare central differences only at interior samples and state the time assigned to each forward-difference estimate. Add angle noise and plot velocity error. Compare two time steps with both noise-free and noisy input.

Save q , dq/dt and absolute-error plots with RMSE over the same time window. Noise-free error should decrease for sufficiently small steps, while differentiating a noisy signal too frequently can worsen the estimate. Remove one sample and use actual timestamps: division by constant dt at that point becomes a deliberate error case. M04 covers filtering in detail.

Task 8. Link-endpoint sensitivity

For $p(q)=(L\cdot\cos q, L\cdot\sin q)$, derive $J(q)=(-L\cdot\sin q, L\cdot\cos q)^T$. With $L=0.12$ m and $q=30^\circ$, estimate $\Delta p \approx J\cdot\Delta q$ for $\Delta q=0.1^\circ, 1^\circ, 10^\circ$. Compare with exact $p(q+\Delta q)-p(q)$; the error magnitude shows where the linear approximation loses accuracy. Deliverables: tangential-displacement diagram and error table in millimetres.

Separately integrate a known velocity with the trapezoidal rule and check that the angle returns after a full period. Calculate work done by a constant torque of 0.2 N·m through 30° : $W=\tau\cdot\Delta q$, with angle in radians. Do not confuse torque with energy even though both have dimensional product N·m: their physical meanings and operations differ.

ODEs: a compliant leg and a free pendulum

Formulating an initial-value problem

For a spring-supported mass with displacement measured from equilibrium, $m \cdot x'' + c \cdot x' + k \cdot x = F(t)$. Introduce $v = x'$ and state $y = (x, v)$; then $x' = v$ and $v' = (F - c \cdot v - k \cdot x)/m$. Initial values $x(0)$ and $v(0)$ are compulsory parts of the problem. Measuring from equilibrium removes constant gravity; with another coordinate definition, explicitly restore gravity with the correct sign.

Explicit Euler evaluates the derivative at the step's start. Semi-implicit Euler updates v first, then uses the new v to update x . RK4 evaluates four intermediate derivatives; implement it for general $y' = f(t, y)$. It has a fixed step and fourth-order accuracy for sufficiently smooth problems; library RK45 uses a different scheme and adaptive control, so it is not the same algorithm.

Task 9. Mass-spring-damper

Use the parameters on page 1, $x(0) = 0.02$ m, $v(0) = 0$ and $F = 0$, for 5 s. Implement the three methods and a reference `solve_ivp` calculation with stricter tolerances. For each method, test steps 0.04; 0.02; 0.01 s and compare x and v at common times. In a separate undamped experiment with $c = 0$, track $E = mv^2/2 + kx^2/2$.

Save state, energy and error-versus-step plots. For $c > 0$ without external force, energy must decrease in the continuous model. Energy growth may indicate numerical instability or a sign error. A decay plot alone is insufficient: check equilibrium $x = v = 0$ and compare with the analytical solution for $c = 0$.

Task 10. Pendulum model of a free link

Set $\theta'' = -(g/L) \cdot \sin\theta$, $L = 0.3$ m, $\theta(0) = 0.1$ rad, $\omega(0) = 0$. Measure the period between zero crossings in the same direction and compare with $T \approx 2\pi \cdot \sqrt{L/g}$. Increase the initial angle to 1 rad and demonstrate the small-angle approximation's limitation. Energy $E = mL^2\omega^2/2 + mgL(1 - \cos\theta)$ must remain constant in the ideal continuous model.

Test $\theta = 0$, a negative initial angle and an excessively large step. Reject negative m or L before integration. Save solver settings and an error estimate; agreement between two equally faulty implementations cannot replace an independent analytical reference. Actual foot impacts, joint friction and backlash are not yet part of this model.

The shared robot-math test bench and error checks

Connect all notebooks through one scenario

Scenario: a virtual encoder produces $u(t)$, calibration converts it to $q(t)$, geometry computes the link endpoint, a numerical derivative gives velocity, and an independent pendulum model supplies reference motion. Define each stage's input/output contract. Data include t_s , $encoder_count$, q_rad , p_x_m and p_z_m ; settings include L_m , $a_rad_per_count$ and b_rad .

First generate a trajectory from the ODE, then convert q to u through inverse calibration and quantisation. The analyser must recover motion with the expected precision loss. Combine existing functions; a separate large project is unnecessary. In the report, distinguish model, quantisation, calibration and numerical-differentiation errors: the same final RMSE can have different causes.

Six required scenarios

1. Wrong units. Pass a length in millimetres to an interface expecting metres; a schema or scale check must explain the failure. **2. Zero vector.** Normalisation must return a clear error rather than propagate NaN through the remaining calculation. **3. Singular calibration.** All u values are equal; the report identifies insufficient rank and does not produce an unsupported scale coefficient.

4. Poor conditioning. Nearly identical equations give a sensitive answer; the programme must not present a small residual as proof of accuracy. **5. Missing time sample.** A removed packet must not cause an artificial doubling of velocity through a hard-coded dt . **6. Unstable step.** Detect energy growth in an ideal spring and reduce it by adjusting the method or step.

Numerical and geometric checks

For exact noise-free examples, combine absolute and relative tolerances: $|a-b| \leq atol + rtol \cdot |b|$. Absolute tolerance matters near zero; relative tolerance matters when scale changes. Choose tolerances before viewing results and label their units. For integration methods, the main check is convergence over a series of steps, rather than one fortuitously accurate value.

In `test_report.csv`, record test name, input scenario, expectation, observation, threshold and pass/fail. Expected failures are valid: a test passes if the programme recognises an invalid formulation. Save at least one plot each of a faulty and corrected version. For the final run, remove computed results and recreate them from raw data using one recorded command sequence.

Step-by-step work: hours 1-40

Block A: coordinate and force mathematics

Hours 1-2. Complete the entry diagnostic, create a quantity log and define axis conventions. Solve two unit-conversion examples and resolve a force into components by hand. Deliverable: assumptions.md with a link diagram and correctness criteria.

Hours 3-4. Study length, normalisation, dot/cross and projections. Calculate torque by hand, then check with a library function. **Hours 5-6.** Study matrices, product order and transposition. Draw a rotation's effect on a basis; explain the result before running code.

Hours 7-8. Study points, directions and shifting the coordinate origin. **Hours 9-10.** Solve a simple force system by hand and check dimensions. If these topics are easy, use a more complex force orientation while retaining the same object and a clear manual check.

Hours 11-20 - practical. First 2 h: tasks 1 and 2 by hand; next 2 h: transformation functions; 2 h: basis and torque plots; 2 h: zero vector, degrees and radians, and reverse transformation; final 2 h: tests and a short explanation without code. Finish when every drawing and number can be related to one chosen coordinate frame.

Block B: calibration and numerical stability

Hours 21-22. Apply Gaussian elimination to support reactions, then repeat with solve. **Hours 23-24.** Study rank and overdetermined systems; construct the calibration matrix on paper. **Hours 25-26.** Study residuals, least squares and QR's role; compare a prediction with the observed point.

Hours 27-28. Explore conditioning with two nearly identical equations and understand solution error versus residual. **Hours 29-30.** Study stiffness-matrix eigenvector directions. Avoid proofs for arbitrary-size matrices until you can explain the 2×2 example.

Hours 31-40 - practical. 2 h: supports and reaction plot; 2 h: encoder generator and calibration; 2 h: held-out validation and a narrow range; 2 h: conditioning and eigenvector directions; 2 h: documenting coefficients and tests. Checkpoint: recover parameters from noise-free data, then deliberately create an unidentifiable calibration.

Before proceeding, write three short answers: why inverse is not a replacement for a normal solve call; why a small residual does not guarantee an accurate parameter; why more identical observations do not resolve insufficient input diversity. If you cannot answer, replace some plot styling with another hand-worked example.

Step-by-step work: hours 41-75

Block C: motion as a function of time

Hours 41-42. Review derivatives and velocity units for linear and angular motion. **Hours 43-44.** Study sin/cos and the chain rule; derive reference q , ω and α . **Hours 45-46.** Study forward and central differences, the time assigned to an estimate and boundary handling.

Hours 47-48. Study integration as accumulation, work done by torque and the trapezoidal rule.

Hours 49-50. Derive partial derivatives and one column of the link-endpoint Jacobian. The derivation is short, so reproduce it unaided.

Hours 51-60 - practical. 2 h: trajectory and exact derivatives; 2 h: finite-difference comparison; 2 h: noise and missing time sample; 2 h: $p(q)$ sensitivity and integral check; 2 h: error plots and conclusions. Do not change noise and step size together without a control run, or you cannot explain the RMSE change.

Deliverable: a table of 'method / step / noise / time window / RMSE / maximum error' and two illustrations: the tangent to the endpoint trajectory and angle recovery by integration. Use three different Δq values for the linear approximation; one small perturbation cannot reveal the model's limit.

Block D: dynamics, integration and presentation

Hours 61-62. Convert the second-order equation into a first-order system; check each term's units. **Hours 63-64.** Write explicit Euler, semi-implicit Euler and RK4 updates, and calculate one step by hand for $x=0.02$ and $v=0$. **Hour 65.** Study local and accumulated error, stability, initial conditions and library-solver tolerances.

Hours 66-75 - practical. 2 h: mass-spring and equilibrium check; 2 h: method and energy comparison; 2 h: pendulum, period and large angle; 2 h: combined encoder→angle→position→velocity processing; 2 h: clean reproducible run, six failures and oral explanation.

Reserve the last 30 minutes for a new variant without copying: change link length, initial angle and step size; predict the direction of period change before running. Then explain what changed in the model and what stayed the same. List two omitted real-leg effects and a measurement that would allow each to be added later.

If integrators take extra time, retain one short file with the shared interface `step(f,t,y,h)` and remove interactive elements. Do not cut force-sign checks, the analytical comparison or the step-reduction experiment: these establish that the mathematical model has been checked, beyond merely running its code.

Acceptance checks and handover to later modules

What to save in robot-math

Four completed notebooks; a function module with clear units; input CSVs; parameter configurations; calibration coefficients; test table; norm, residual, derivative and energy plots; and README with execution order. State the formula and seed for synthetic data; store reference values separately. Label the 'raw measurements' field synthetic until a real sensor is used.

Begin each notebook with a robotics question and end with an engineering conclusion. 'The code ran' is not a conclusion. Suitable statements include: 'the length-scale error increased torque a thousandfold'; 'the narrow encoder range prevents stable scale estimation'; 'the chosen Euler step increases this oscillator's energy'. Fill in all numerical results after your own run.

Passing criteria

1. Each of ten practical tasks has an input, result, independent check and edge case. 2. Parameters, columns and plot axes have units. 3. Analytical and numerical answers are compared over the same time interval. 4. ODE results cover at least three step sizes and both objects. 5. Singularity and instability are explicitly identified. 6. A clean run reproduces conclusions without manually editing intermediate files.

Passing many tests does not replace understanding. For the oral review, choose six questions below at random and answer each with a small diagram or calculation. You may consult NumPy syntax; explain the underlying torque, linear-calibration and ODE formulae independently.

Self-check questions

Why does shifting the coordinate origin affect a point but not a direction? When is $\mathbf{r} \times \mathbf{F}$ zero? What does a negative support reaction mean? How does an overdetermined system differ from an inconsistent one? Why are residual and coefficient error different quantities? How do units affect numerical conditioning?

How do you obtain link-endpoint velocity from joint velocity? Why does numerical differentiation amplify noise? What is a 'linear approximation in angle'? Which initial conditions does a pendulum require? Why is energy conservation useful as a test? How does fixed-step RK4 differ from RK45 in `solve_ivp`?

Subsequent use

M03 uses the same calibration and Jacobian for uncertainty estimation. M04 uses the noisy encoder and derivative to study filtering. M05 uses torque, reactions and eigenvector directions as preparation for mechanics. Preserve the functions' public interface, but do not claim real-robot accuracy from synthetic data. Physical test-bench validation follows electronics and mechanical preparation.

Assigned reading: a short route

Read only the specified passages before the corresponding experiment. References are mainly in English; keep explanations, glossary and report in English. Reading and preparation fit within the existing 75 hours; whole books are not required.

Core reading

1. Gilbert Strang, MIT 18.06: Linear Algebra. [Lectures 1-2, 16, 21](#). Equation geometry and elimination support reaction calculations; projections and least squares support calibration; eigenvector directions support task 6. After each section, solve one 2×2 example by hand.

2. Gilbert Strang, Edwin Herman, Calculus Volume 1 (OpenStax). [§3.1-3.6](#): derivative definition, rules, rate of change, trigonometry and chain rule; [§5.2-5.3](#): definite integral and fundamental theorem. Use in tasks 7-8 for link velocity and angle recovery.

3. Haynes Miller, Arthur Mattuck, MIT 18.03: Differential Equations. [Lecture notes 2, 11-12](#): Numerical methods; Modes and the characteristic polynomial; Good vibrations, damping conditions. For tasks 9-10, explain integration step size, natural frequency and damping effects.

References for implementation

4. Kevin Lynch, Frank Park, Modern Robotics - reference. [§3.2.1 Rotation Matrices, Part 1](#). For task 2, reconstruct the geometric meaning of matrix columns and $R^T R = I$; skip exponential coordinates for now.

NumPy linear algebra. [lstsq: Parameters, Returns](#) and [cond: norms](#). Check A,b shapes, rank and residual meaning; an absent residuals array does not establish exact calibration.

SciPy integration. [solve_ivp: y0, t_eval, rtol, atol, success](#); [RK45: method description](#). In tasks 9-10, distinguish output times, internal RK45 steps and your own constant-step RK4 implementation.

Minimum reading deliverable. Four sheets: change of basis; calibration with residuals; derivative and integral; ODE with initial conditions. Each includes your own formula, units and one independent check.

Glossary: terms and definitions

Vector. A geometric quantity with direction and magnitude; its components depend on the chosen basis, while the quantity itself does not.

Basis. A set of linearly independent vectors in terms of which every vector in the space has a unique representation.

Coordinate frame. An origin together with a basis; point coordinates change when the origin shifts or the axes rotate.

Rotation matrix. A matrix describing an orthonormal basis's orientation; a proper rotation satisfies $R^T R = I$ and $\det R = 1$.

Cross product. In three dimensions, a vector perpendicular to both operands; its direction follows the right-hand rule and its magnitude is the parallelogram's area.

Torque. The vector $\mathbf{r} \times \mathbf{F}$ about a chosen point; it describes a force's rotational effect and is measured in $\text{N} \cdot \text{m}$.

Rank. Number of independent matrix columns or rows; it determines how many independent constraints a linear system contains.

Residual. The difference $\mathbf{A} \cdot \mathbf{x} - \mathbf{b}$ for a proposed solution; a small residual alone does not guarantee accurate unknown parameters.

Least squares. Choosing parameters to minimise the sum of squared residuals; fits a model to redundant, usually inconsistent measurements.

Conditioning. Sensitivity of a problem's result to small input changes; depends on its formulation, units and scaling.

Eigenvector direction. Direction of a nonzero vector $\mathbf{v}$ satisfying $\mathbf{A} \cdot \mathbf{v} = \lambda \mathbf{v}$. The matrix scales the vector while preserving the line on which it lies.

Derivative. Limit of the ratio of function increment to argument increment; differentiating position with respect to time gives velocity in m/s .

Integral. Accumulated contribution of a function over an interval; integrating velocity over time gives displacement with consistent initial conditions.

Jacobian. Matrix of partial derivatives of output coordinates with respect to inputs; approximately maps small input changes to output changes.

Ordinary differential equation (ODE). Equation relating an unknown function of one variable to its derivatives; a motion model also requires initial conditions.

Time step. Time interval for one numerical state update; affects trajectory error, stability and computation cost.

Numerical stability. Property of a numerical scheme in which perturbations do not grow unacceptably for the problem and chosen integration step.

Equipment, installation and checks

Complete preparation within the first study blocks in place of another general overview. The budget remains 75 hours; delivery waiting time is excluded. Statuses below specify actions for you to take, not purchases or installations already completed.

Equipment and purchases

Use the MacBook Pro M4, editor, Git, uv/Python, NumPy, Matplotlib and pytest from M01. **No new purchases.** All M02 objects are computational: no motor, IMU, electronics or mechanical parts are needed. Books are available through the supplied links; print copies are optional.

Install now: macOS ARM

Add **SciPy** to robot-math if absent: [official installation guide](#). This is M02's addition for ODEs; the earlier 'SciPy from M01' wording means using the same environment, not an earlier mandatory installation. Pin its version and lockfile.

For the four notebooks, add **JupyterLab and ipykernel** if the editor does not already open notebooks: [Jupyter installation](#). The notebook kernel must use project Python. **SymPy is optional**, only for checking symbolic derivatives: [official instructions](#). Hand derivation remains part of the task.

Installation checks

1. Record Python path, arm64 architecture and library versions. 2. Run `import numpy, scipy, matplotlib`; open a notebook with the project kernel. 3. Solve a 2×2 system and check $A \cdot x = b$. 4. Integrate $y' = -y$, $y(0) = 1$ to $t = 1$ and compare with $\exp(-1)$. 5. Restart the kernel and run all cells in order.

Check numerical error with explicit `rtol/atol` and a comparison tolerance. Save one PNG without an interactive window, one CSV and a pytest result. This establishes environment readiness; verify each model's accuracy separately in the tasks.

Optional work and later modules

You may repeat the short calculation in M01's Ubuntu 24.04 ARM64 in UTM if using that environment. Main work runs natively on the Mac. M02 requires no ROS 2, Gazebo or GPU; do not create a separate numerical-library environment in the VM without a task that needs it.

Ready for M03: dependencies are pinned, notebook→CSV→plot is saved, and robot-math runs cleanly. If installation takes too long, reduce plot styling while retaining mathematical checks and the overall budget.

Motion fundamentals: from joint angle to foot velocity

Compulsory, within 75 hours. Use 1 h from session 49-50 and 2 h of practice 51-60 already allocated to p(q) sensitivity and checks. This connects tasks 2 and 8; it does not add tasks beyond the plan. Before starting, convert 30° to $\pi/6$ rad and explain why m/s differs from rad/s.

Why joint velocity differs from point velocity

Consider a rigid link of length $L=0.12$ m with one revolute joint. Frame A is fixed: its origin is on the joint axis, x points right, y upwards, and positive q is anticlockwise from x. The foot point has coordinates $p_A(q)=(L \cos q, L \sin q)$. Length is constant; backlash and deformation are excluded. This requires geometry, not link mass.

For a small rotation, endpoint displacement is approximately tangent to the circle. By the chain rule, $v_A=dp_A/dt=J(q)\cdot\dot{q}$, where $J=(-L \sin q, L \cos q)^T$, in m/rad. Thus $|v_A|=L|\dot{q}|$ and $p_A\cdot v_A=0$. The Jacobian depends on configuration: equal angular velocities at different q give equal endpoint speed but different directions. Even at constant $\dot{q}$, velocity changes direction: acceleration $a_A=-p_A\cdot\dot{q}^2$ points towards the joint. This last equality assumes $\ddot{q}=0$.

Numerical example and change of fixed frame

At $q=\pi/6$ rad and $\dot{q}=0.5$ rad/s, $p_A=(0.103923; 0.060000)$ m, $v_A=(-0.030000; 0.051962)$ m/s and $|v_A|=0.060000$ m/s. Over a short time $\Delta t=0.01$ s, tangential displacement is approximately 0.6 mm; distinguish distance travelled in this interval from a change in one coordinate.

Let $R_{BA}=[[0,-1],[1,0]]$ describe A's axes in another fixed frame B, and $t_{BA}=(0.20;0.10)$ m be A's origin position in B. Then $p_B=R_{BA}\cdot p_A+t_{BA}=(0.140000;0.203923)$ m, but $v_B=R_{BA}\cdot v_A=(-0.051962;-0.030000)$ m/s. The derivative of a constant translation is zero. A moving B introduces additional terms; that case is not modelled here.

Two checks using existing functions

1. In robot-math, compute the central difference $[p(q+\dot{q}h)-p(q-\dot{q}h)]/(2h)$, $h=10^{-4}$ s, and compare with $J\dot{q}$. For these noise-free float64 values, use a teaching tolerance of 10^{-8} m/s on the error norm. Save a table for $q=0, \pi/6, \pi/2$; check tangency, speed magnitude and inverse coordinate transformation.

2. On the leg diagram, label angle, both frames and the v arrow. Predict doubled speed when L doubles and a reversed arrow when $\dot{q}$ changes sign. Deliverables: one drawing and tests for tasks 2/8. These check kinematics; they do not establish the existence of a motor capable of producing the motion.

Read before calculating: Lynch, Park, Modern Robotics: [chapter 5 introduction, deriving \$\dot{x}=J\dot{q}\$; §3.3.1, changing coordinate frames](#). Spatial velocities are studied in M08.